



Sistemas Robóticos

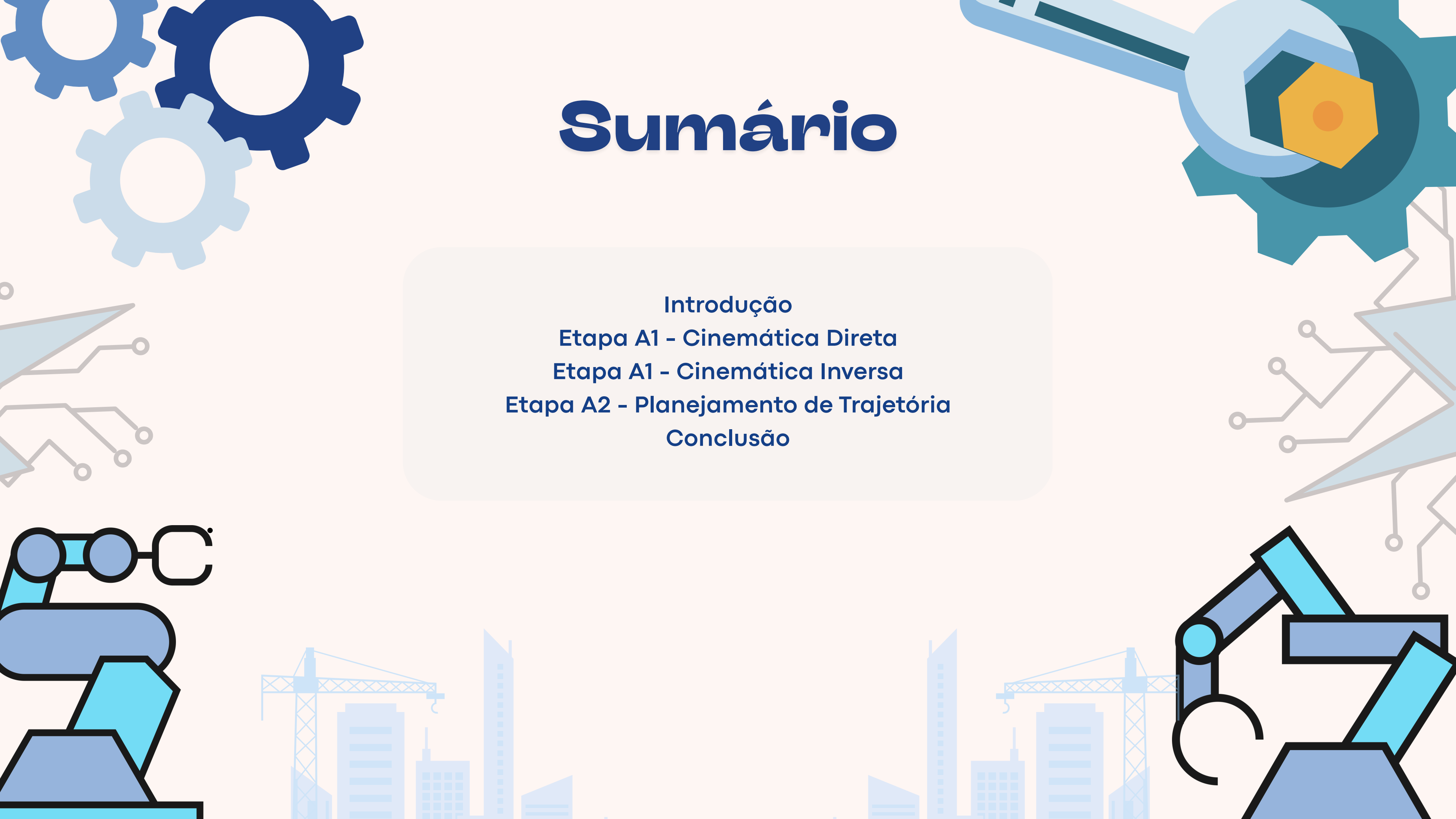
Modelagem e Simulação do robô UR5



Equipe

Ludmila Nascimento

Rafael Dias Heydt



Sumário

Introdução

Etapa A1 - Cinemática Direta

Etapa A1 - Cinemática Inversa

Etapa A2 - Planejamento de Trajetória

Conclusão

Introdução

- O trabalho foca no desenvolvimento e na validação de modelos matemáticos para descrever a estrutura e orientar os movimentos do robô UR5.
- A modelagem aborda a cinemática direta e a cinemática inversa.
- Os algoritmos de cinemática e o planejamento de trajetória usando polinômios de quinto grau são validados por meio de simulações no software CoppeliaSim.



CARACTERÍSTICAS DO ROBÔ UR5

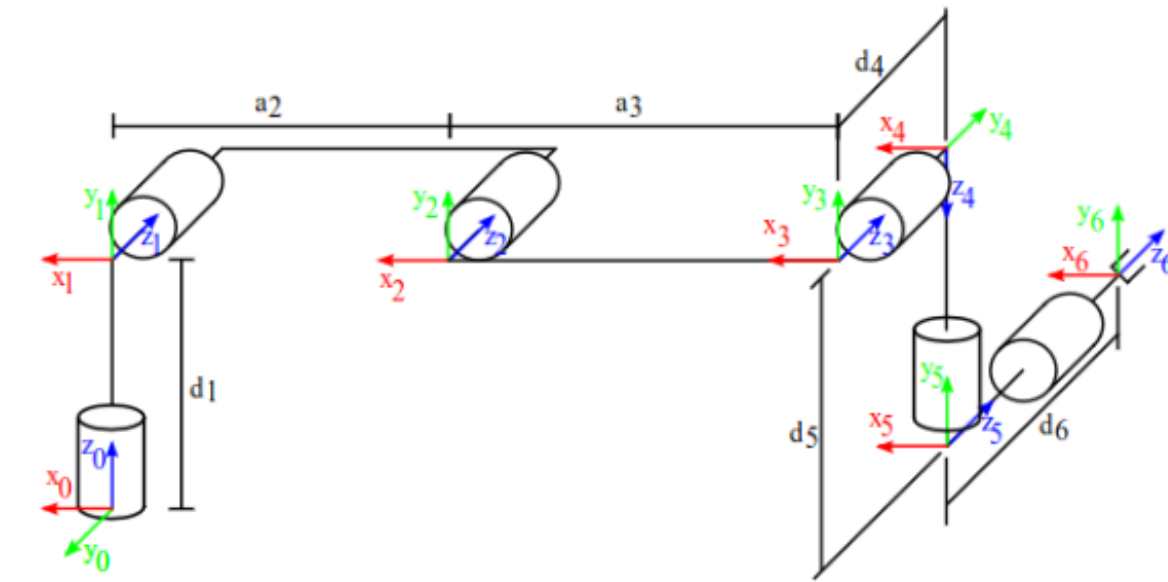
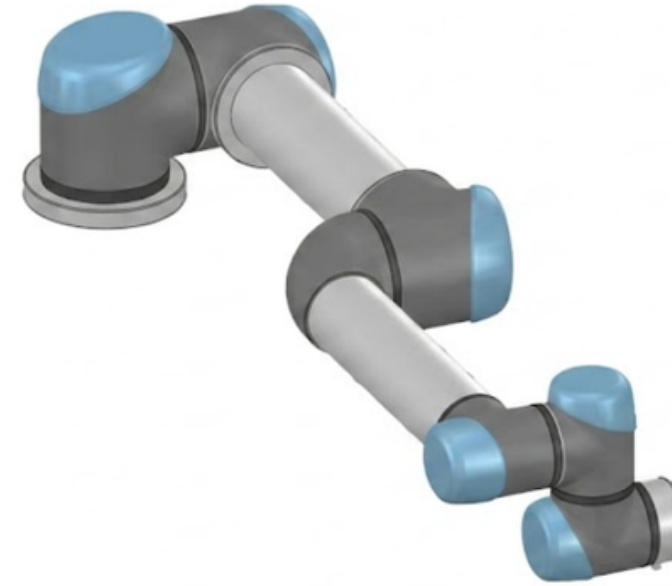
Características	Descrição
Graus de Liberdade	6 juntas (6 DoF)
Peso	18,4 kg
Capacidade de carga	5 kg
Alcance	850 mm
Repetibilidade	$\pm 0,03$ mm
Consumo de energia	Aproximadamente 250W

Cinemática Direta

Sequência de Transformações

$$T_6^0(\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6) = T_1^0(\theta_1)T_2^1(\theta_2)T_3^2(\theta_3)T_4^3(\theta_4)T_5^4(\theta_5)T_6^5(\theta_6)$$

$$T_j^i = \begin{bmatrix} R_j^i & \vec{P}_j^i \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} x_x & y_x & z_x & (P_j^i)_x \\ x_y & y_y & z_y & (P_j^i)_y \\ x_z & y_z & z_z & (P_j^i)_z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$



Notação de Denavit-Hartenberg

$$T_i^j = \underbrace{\begin{bmatrix} c\theta_i & -s\theta_i & 0 & 0 \\ s\theta_i & c\theta_i & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{\text{Rot}_{z,\theta_i}} \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{\text{Trans}_{z,d_i}} \underbrace{\begin{bmatrix} 1 & 0 & 0 & a_i \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{\text{Trans}_{x,a_i}} \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & c\alpha_i & -s\alpha_i & 0 \\ 0 & s\alpha_i & c\alpha_i & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}}_{\text{Rot}_{x,\alpha_i}} = \begin{bmatrix} c\theta_i & -s\theta_i c\alpha_i & s\theta_i s\alpha_i & a_i c\theta_i \\ s\theta_i & c\theta_i c\alpha_i & -c\theta_i s\alpha_i & a_i s\theta_i \\ 0 & s\alpha_i & c\alpha_i & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Junta	a (m)	α	d (m)	θ
1	0	$+90^\circ$	0.089159	θ_1
2	-0.425	0°	0	θ_2
3	-0.39225	0°	0	θ_3
4	0	$+90^\circ$	0.10915	θ_4
5	0	-90°	0.09465	θ_5
6	0	0°	0.0823	θ_6

Cinemática Direta

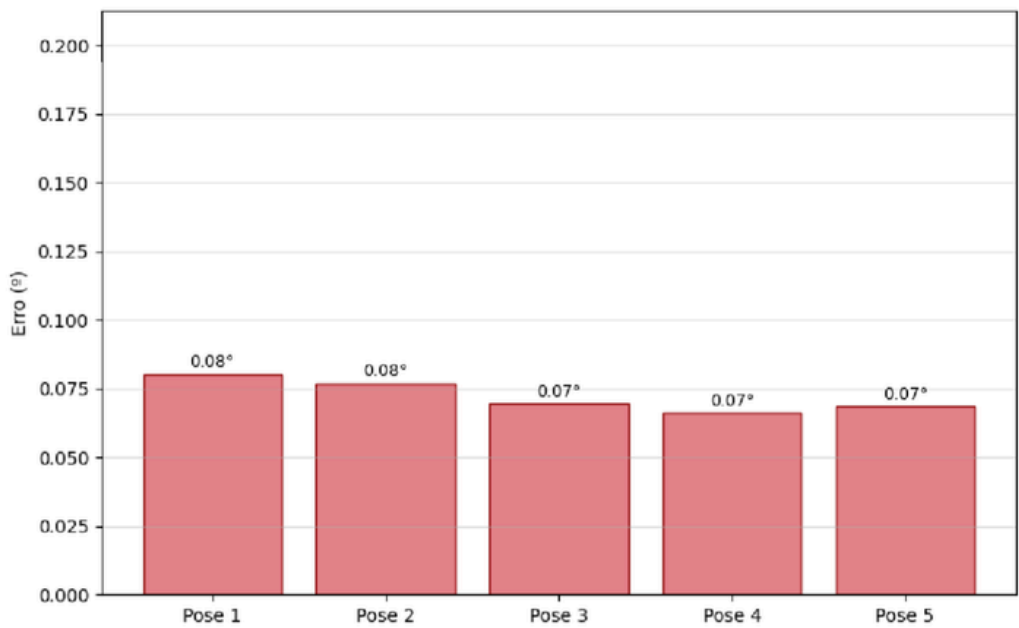
Erro de Posição

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_2)^2 + (z_2 - z_1)^2}$$

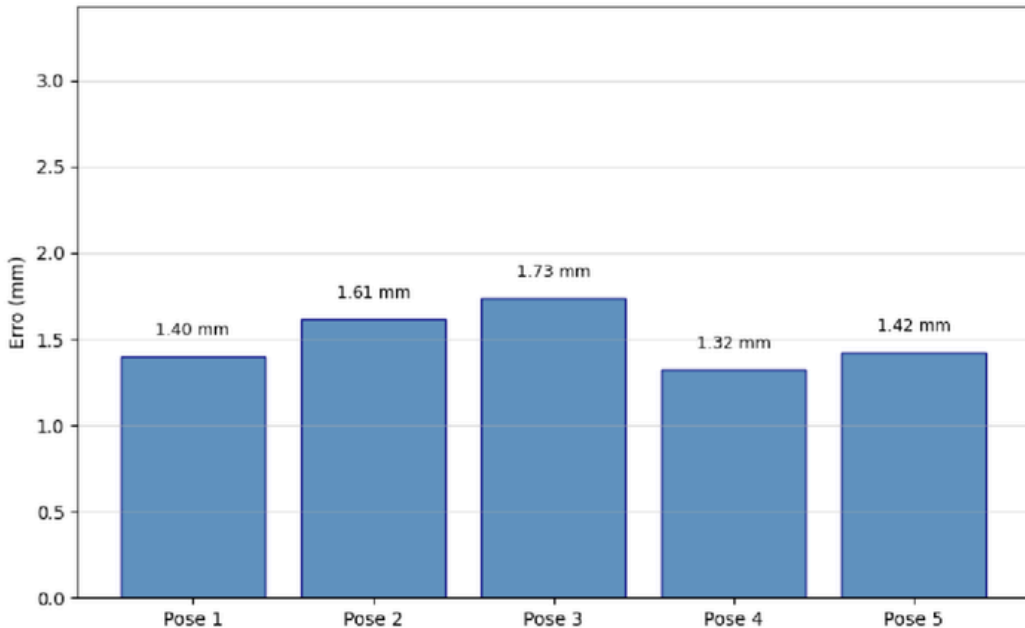
Erro de Orientação

$$e_{ang} = \arccos \left(\frac{\text{tr}(R_{calc}^T \cdot R_{gt}) - 1}{2} \right)$$

Erro de Orientação da garra



Erro de Posição da garra



Comparação de posições e ângulos cinemática direta calculado vs simulados

Posição	Ângulos das Juntas(°)	Posição calculada (x, y, z)(m)	Posição CoppeliaSim (x, y, z)(m)	Ângulos de Euler Calculados ((φ, θ, ψ)) (°)	Ângulos de Euler CoppeliaSim ((φ, θ, ψ)) (°)
1	(0, -45, 90, -45, 30, 0)	(0.1111, -0.2615, 0.7617)	(0.1111, -0.2629, 0.7616)	(-90, 0, -150)	(-90.08, -0, -150)
2	(60, -30, 25, -45, 65, 30)	(0.3697, 0.2734, 0.7867)	(0.3712, 0.2729, 0.7863)	(-134.02, 2.35, -68.23)	(-134.09, 2.34, -68.25)
3	(-60, -30, 75, -45, 90, -30)	(-0.2466, 0.0886, 0.6004)	(-0.2477, 0.0873, 0.5999)	(105.44, 23.97, -122.95)	(105.51, 23.95, -122.91)
4	(30, -60, -45, -30, 45, 45)	(0.6735, 0.1192, 0.4841)	(0.6742, 0.1183, 0.4835)	(-144.74, -30, -114.74)	(-144.81, -30.01, -114.69)
5	(90, -45, 60, 30, -60, 90)	(0.1971, 0.0243, 0.7278)	(0.1980, 0.0253, 0.7273)	(150, 45, 90)	(150.08, 44.97, 90.03)

```
def dh_transform(alpha, a, d, theta):  
    """  
    Transformação DH:  
    T = Rz(theta) · Tz(d) · Tx(a) · Rx(alpha)  
    """  
    ct, st = np.cos(theta), np.sin(theta)  
    ca, sa = np.cos(alpha), np.sin(alpha)  
    return np.array([  
        [ct, -st*ca, st*sa, a*ct],  
        [st, ct*ca, -ct*sa, a*st],  
        [ 0, sa, ca, d ],  
        [ 0, 0, 0, 1 ]  
    ])  
  
def fk(thetas_deg):  
    """FK completa. Retorna T (4x4) e cadeia de frames."""  
    thetas = np.radians(thetas_deg)  
    T = np.eye(4)  
    chain = [T.copy()]  
    for (alpha,a,d), theta in zip(DH, thetas):  
        T = T @ dh_transform(alpha, a, d, theta)  
        chain.append(T.copy())  
    return T, chain
```

Cinemática Inversa

```
def ik(T06_desired):  
    T06 = np.array(T06_desired, dtype=float)  
    solutions = []  
  
    for idx1, t1 in enumerate(_solve_theta1(T06)):  
        shoulder = 'left ' if idx1 == 0 else 'right'  
  
        for idx5, t5 in enumerate(_solve_theta5(T06, t1)):  
            wrist = 'down' if idx5 == 0 else 'up '  
  
            t6 = _solve_theta6(T06, t1, t5)  
            T14 = _compute_T14(T06, t1, t5, t6)  
  
            for idx3, t3 in enumerate(_solve_theta3(T14)):  
                elbow = 'up ' if idx3 == 0 else 'down'  
  
                t2 = _solve_theta2(T14, t3)  
                t4 = _solve_theta4(T14, t2, t3)  
  
                sol = np.degrees([t1, t2, t3, t4, t5, t6])  
                solutions.append(sol)  
  
    return solutions
```

$$\theta_1 = \text{atan2} \left((P_5^0)_y, (P_5^0)_x \right) \pm \arccos \left(\frac{d_4}{\sqrt{(P_5^0)_x^2 + (P_5^0)_y^2}} \right) + \frac{\pi}{2}$$

$$\theta_5 = \pm \arccos \left(\frac{(P_6^1)_z - d_4}{d_6} \right)$$

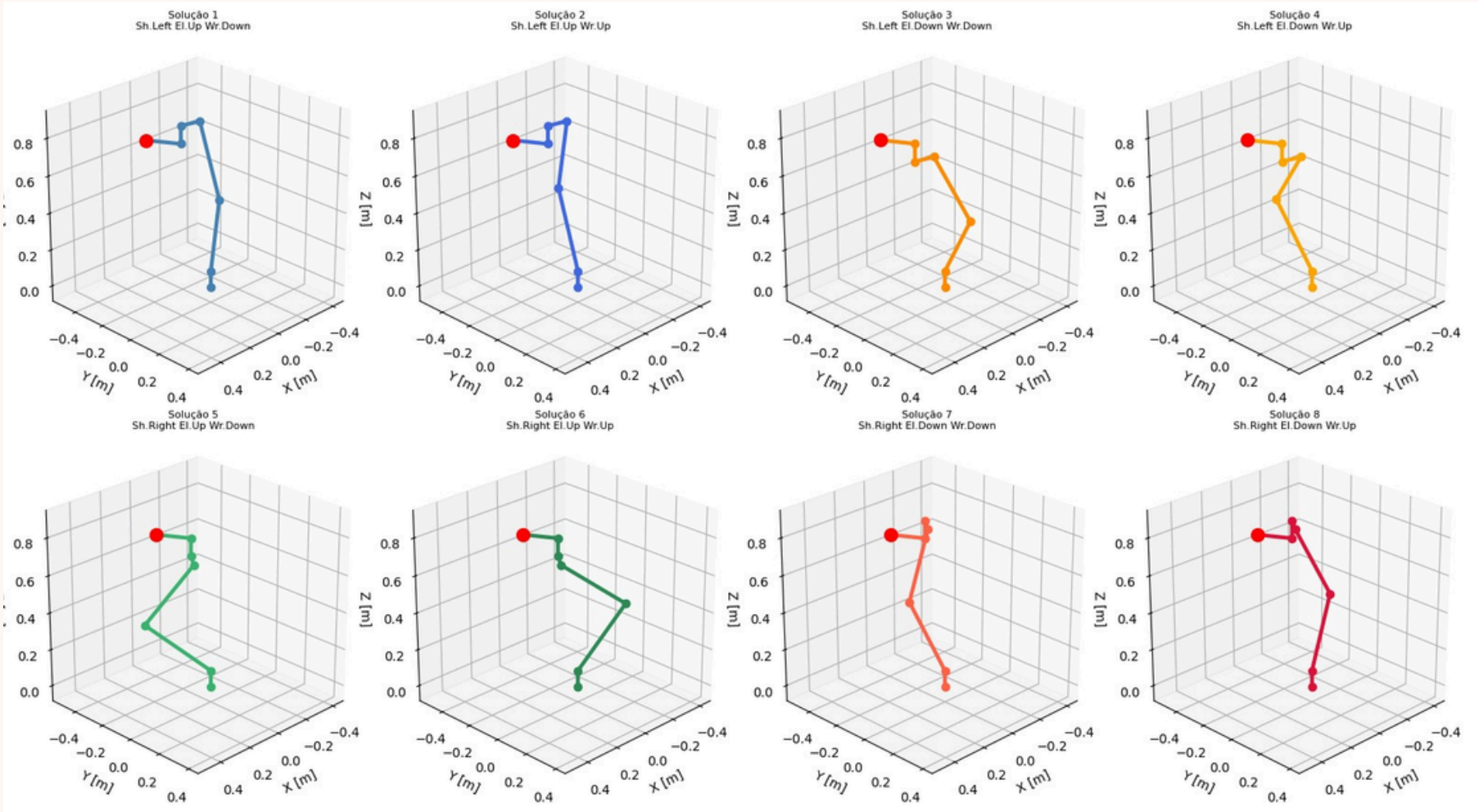
$$\theta_6 = \text{atan2} \left(\frac{-(T_1^6)_{zy}}{\sin \theta_5}, \frac{(T_1^6)_{zx}}{\sin \theta_5} \right)$$

$$\theta_3 = \pm \arccos \left(\frac{\|\vec{P}_3^1\|^2 - a_2^2 - a_3^2}{2 a_2 a_3} \right)$$

$$\theta_2 = -\text{atan2} \left((P_3^1)_y, -(P_3^1)_x \right) + \arcsin \left(\frac{a_3 \sin(\theta_3)}{\|\vec{P}_3^1\|} \right)$$

$$\theta_4 = \text{atan2} \left((T_4^3)_{xy}, (T_4^3)_{xx} \right)$$

Cinemática Inversa



Validação da cinemática inversa:
Pose Alvo e Erro Angular

Teste	Posição Alvo	Orientação Alvo (Euler)	Ângulos Calculados (IK) (°)	Ângulos Originais (°)	$\Delta\theta$ Médio (°)
	(x, y, z) (m)	(ϕ, θ, ψ) (°)	($\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6$)	($\theta_1, \theta_2, \theta_3, \theta_4, \theta_5, \theta_6$)	
1	(0.1111, -0.2629, 0.7616)	(-90.08, 0.00, -150.00)	(-3.08, -45.52, 89.92, -44.55, 33.08, 0.12)	(0, -45, 90, -45, 30, 0)	1.22
2	(0.3712, 0.2729, 0.7863)	(-134.09, 2.33, -68.26)	(59.72, -30.05, 24.95, -44.88, 65.19, 29.81)	(60, -30, 25, -45, 65, 30)	0.15
3	(-0.2477, 0.0873, 0.5999)	(105.51, 23.95, -122.91)	(-57.90, -29.69, 74.97, 149.61, 72.02, -29.43)	(-60, -30, 75, 150, 70, -30)	0.90
4	(0.6742, 0.1183, 0.4835)	(-144.81, -30.01, -114.69)	(29.89, -25.05, -45.03, -19.85, 45.05, 44.83)	(30, -25, -45, -20, 45, 45)	0.09
5	(0.1980, 0.0253, 0.7273)	(150.08, 44.97, 90.03)	(89.53, -45.10, 59.86, 30.47, -59.73, 89.59)	(90, -45, 60, 30, -60, 90)	0.31

Planejamento de Trajetória

- Método do Polinímio de quinto grau:
 - Aproximação da posição das N juntas por um polinómio de quinto grau

$$Ax = b$$

$$q(t_0) = a_0 + a_1 t_0 + a_2 t_0^2 + a_3 t_0^3 + a_4 t_0^4 + a_5 t_0^5 = q_0$$

$$\dot{q}(t_0) = a_1 + 2a_2 t_0 + 3a_3 t_0^2 + 4a_4 t_0^3 + 5a_5 t_0^4 = v_0$$

$$\ddot{q}(t_0) = 2a_2 + 6a_3 t_0 + 12a_4 t_0^2 + 20a_5 t_0^3 = \alpha_0$$

$$q(t_f) = a_0 + a_1 t_f + a_2 t_f^2 + a_3 t_f^3 + a_4 t_f^4 + a_5 t_f^5 = q_f$$

$$\dot{q}(t_f) = a_1 + 2a_2 t_f + 3a_3 t_f^2 + 4a_4 t_f^3 + 5a_5 t_f^4 = v_f$$

$$\ddot{q}(t_f) = 2a_2 + 6a_3 t_f + 12a_4 t_f^2 + 20a_5 t_f^3 = \alpha_f$$

$$\begin{bmatrix} 1 & t_0 & t_0^2 & t_0^3 & t_0^4 & t_0^5 \\ 0 & 1 & 2t_0 & 3t_0^2 & 4t_0^3 & 5t_0^4 \\ 0 & 0 & 2 & 6t_0 & 12t_0^2 & 20t_0^3 \\ 1 & t_f & t_f^2 & t_f^3 & t_f^4 & t_f^5 \\ 0 & 1 & 2t_f & 3t_f^2 & 4t_f^3 & 5t_f^4 \\ 0 & 0 & 2 & 6t_f & 12t_f^2 & 20t_f^3 \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \\ a_4 \\ a_5 \end{bmatrix} = \begin{bmatrix} q_0 \\ v_0 \\ \alpha_0 \\ q_f \\ v_f \\ \alpha_f \end{bmatrix}$$

Planejamento de Trajetória

```
# Sistema linear  $Ax = b$  – coeficientes do polinômio de quinto grau para cada junta
A = np.array([
    [1, t0, t0**2, t0**3, t0**4, t0**5 ],
    [0, 1, 2*t0, 3*t0**2, 4*t0**3, 5*t0**4 ],
    [0, 0, 2, 6*t0, 12*t0**2, 20*t0**3 ],
    [1, tf, tf**2, tf**3, tf**4, tf**5 ],
    [0, 1, 2*tf, 3*tf**2, 4*tf**3, 5*tf**4 ],
    [0, 0, 2, 6*tf, 12*tf**2, 20*tf**3 ],
])
b = np.column_stack([q0, v0, a0, qf, vf, af]) # shape (6, 6)
x = [np.linalg.solve(A, b[i]) for i in range(6)]
```

```
while (t_now - t_start) < tf:
    t_sim = t_now - t_start

    thetas_t = np.zeros(6) # ângulos no instante t
    for j in range(6):
        a = x[j]

        # Posição calculada pelo polinômio em t_sim [rad]
        pos_j = (a[0] + a[1]*t_sim + a[2]*t_sim**2 + a[3]*t_sim**3 + a[4]*t_sim**4 + a[5]*t_sim**5)
        vel_j = (a[1] + 2*a[2]*t_sim + 3*a[3]*t_sim**2 + 4*a[4]*t_sim**3 + 5*a[5]*t_sim**4)
        acc_j = (2*a[2] + 6*a[3]*t_sim + 12*a[4]*t_sim**2 + 20*a[5]*t_sim**3)
        jk_j = (6*a[3] + 24*a[4]*t_sim + 60*a[5]*t_sim**2)

        sim.setJointTargetPosition(joint_handles[j], pos_j)
```

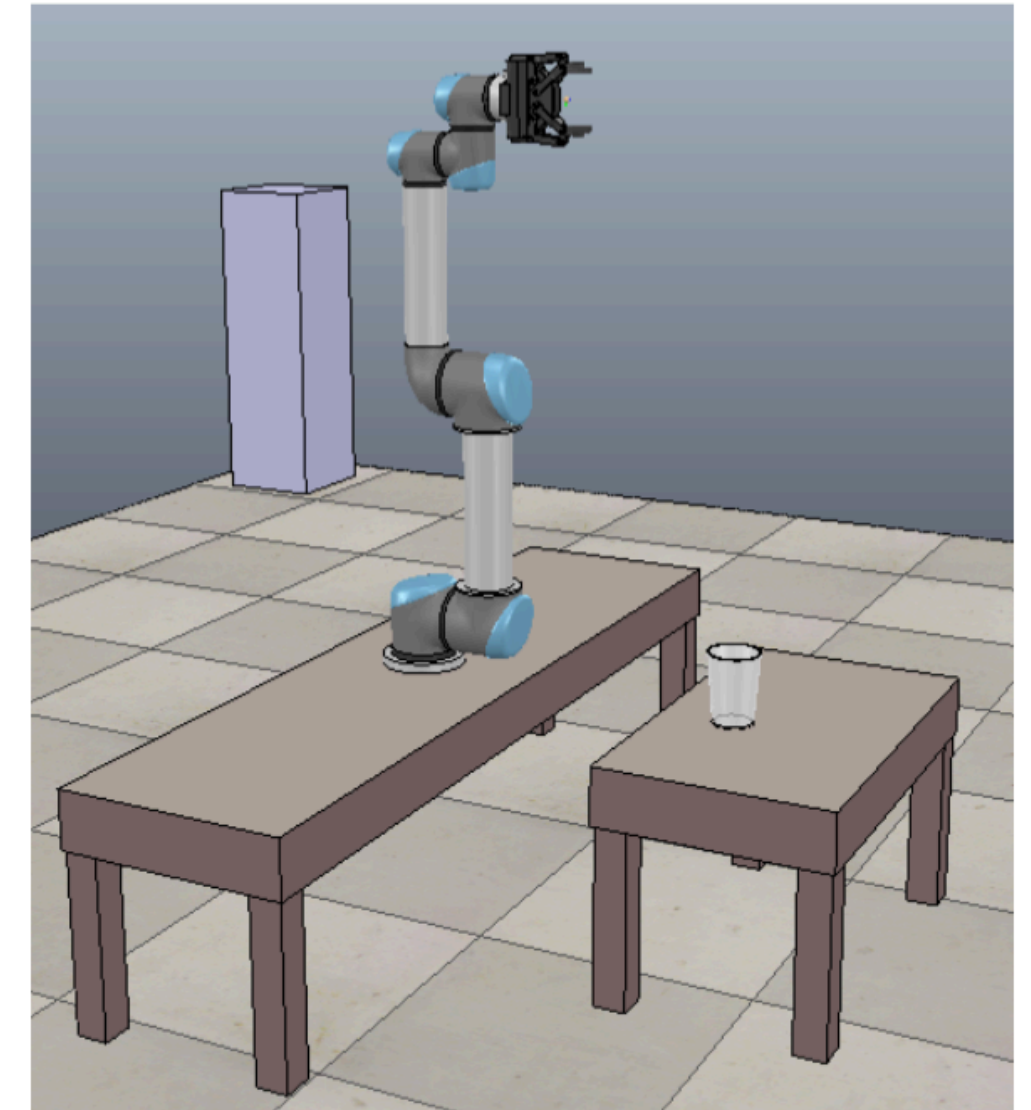
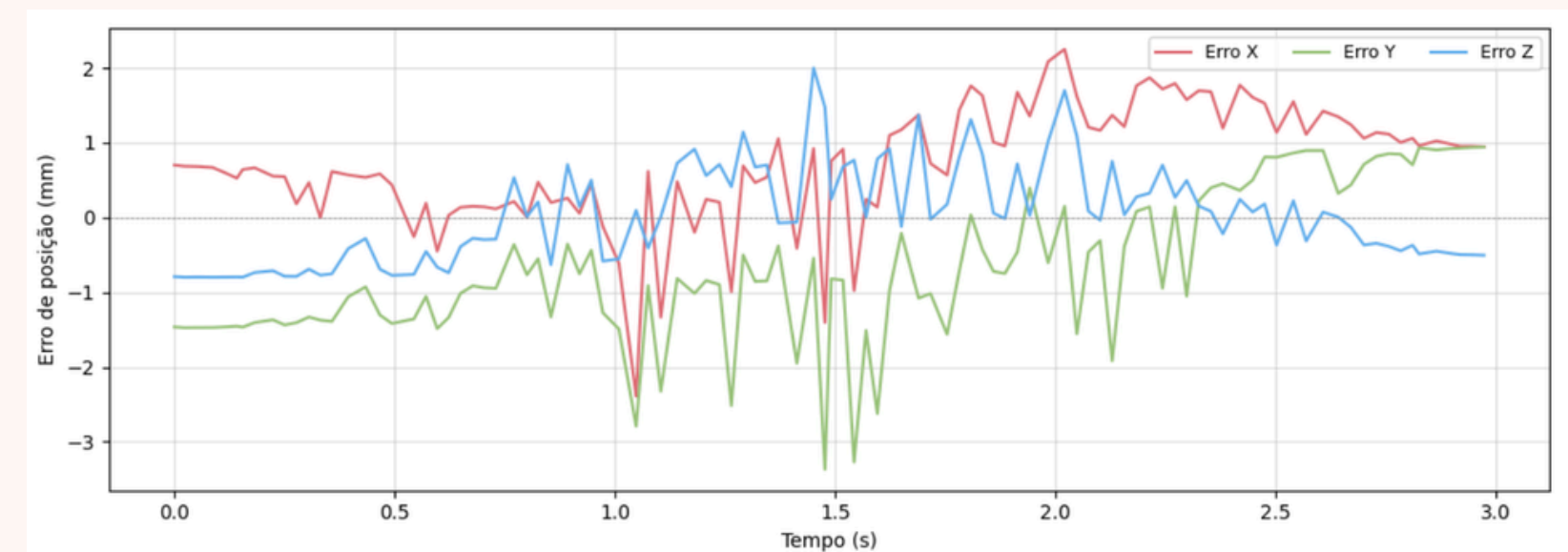
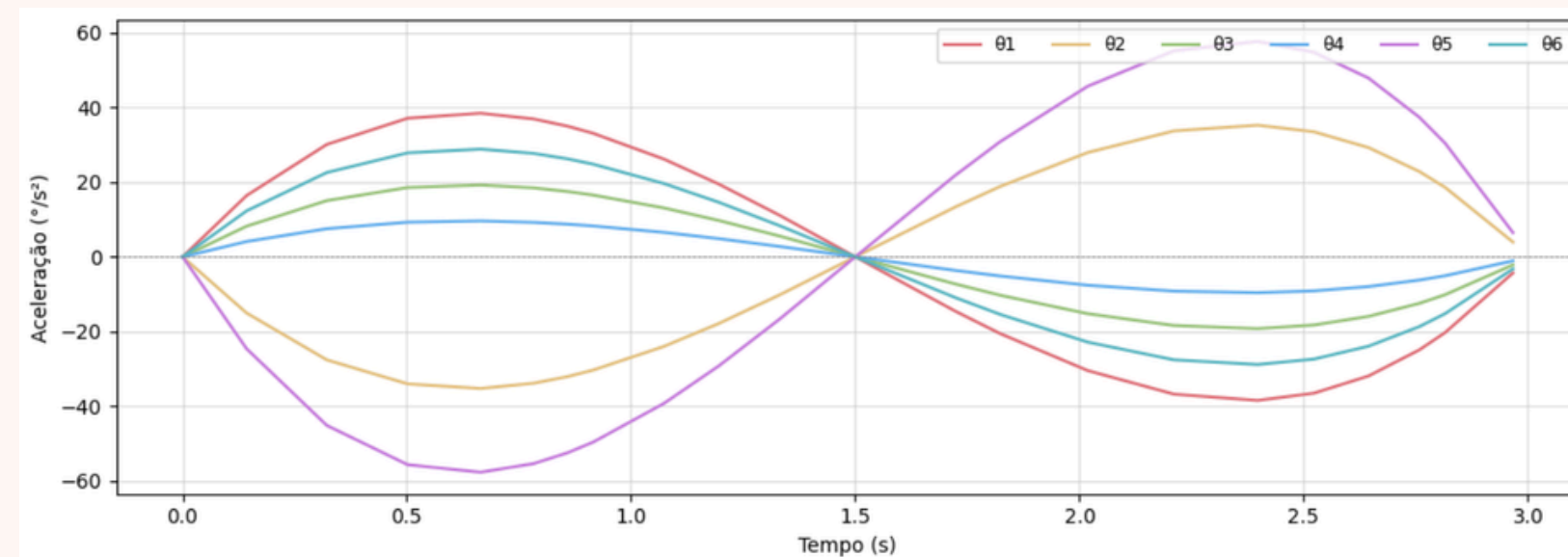
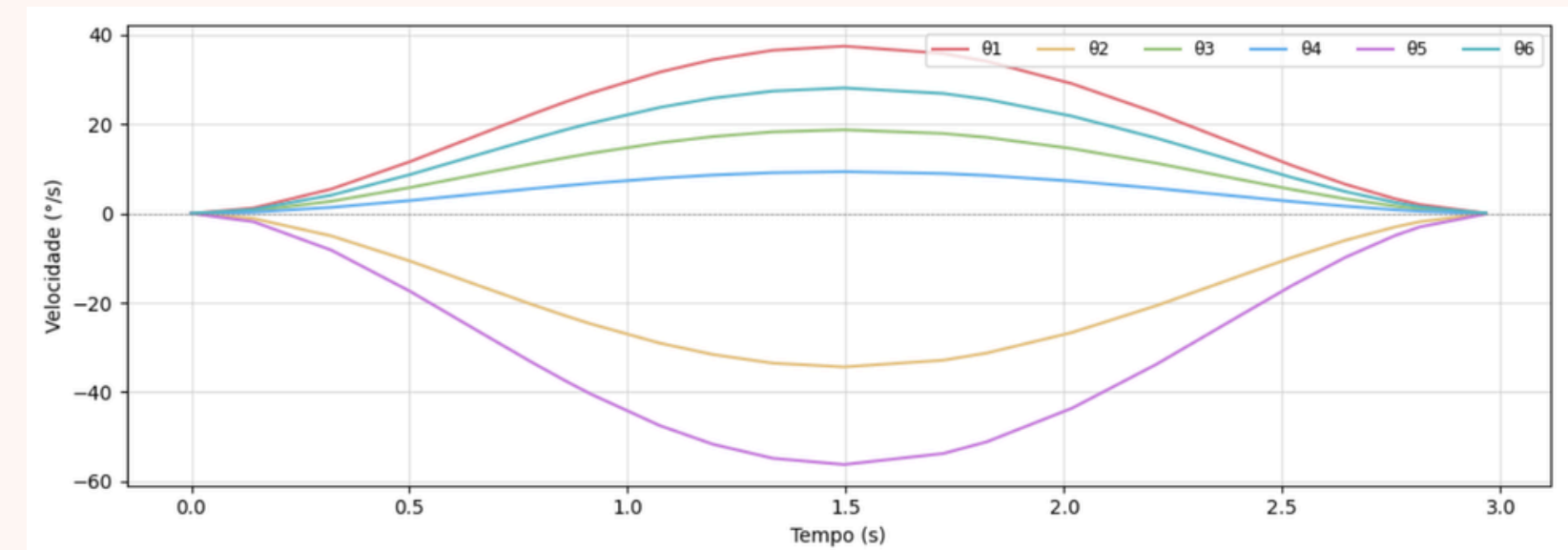
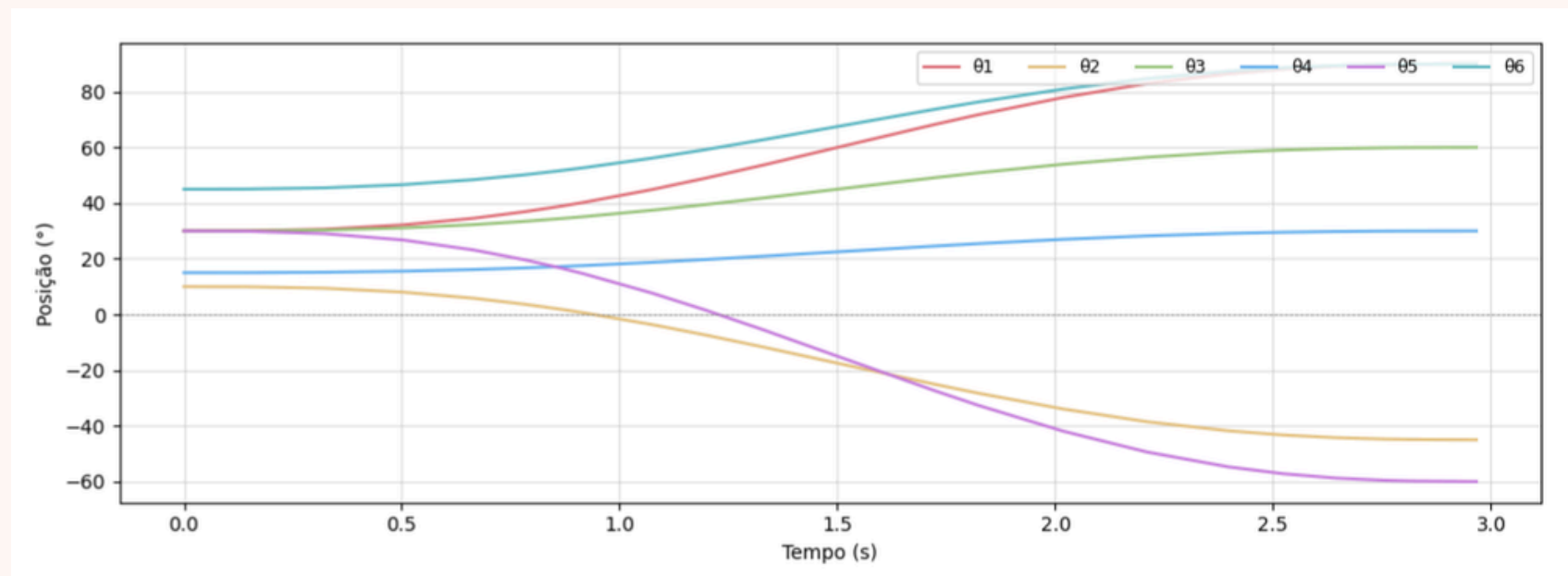


Fig. 3. Cenário no CoppeliaSim

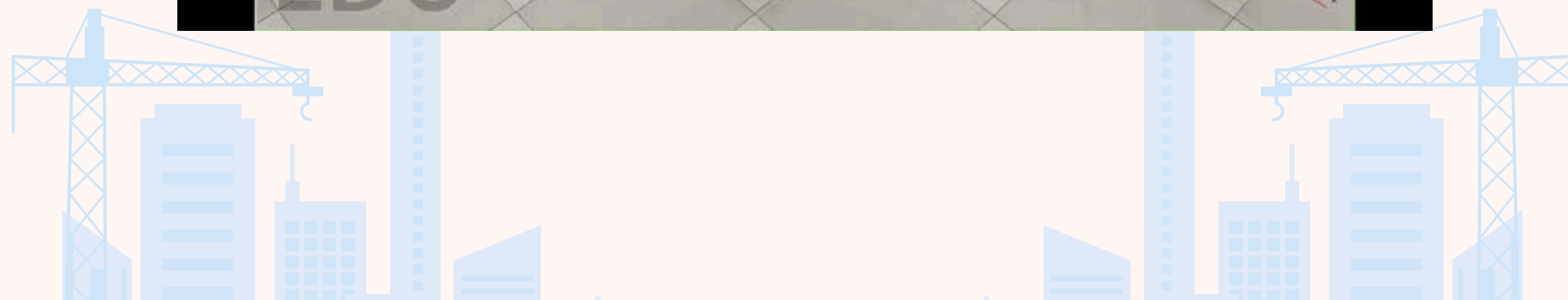
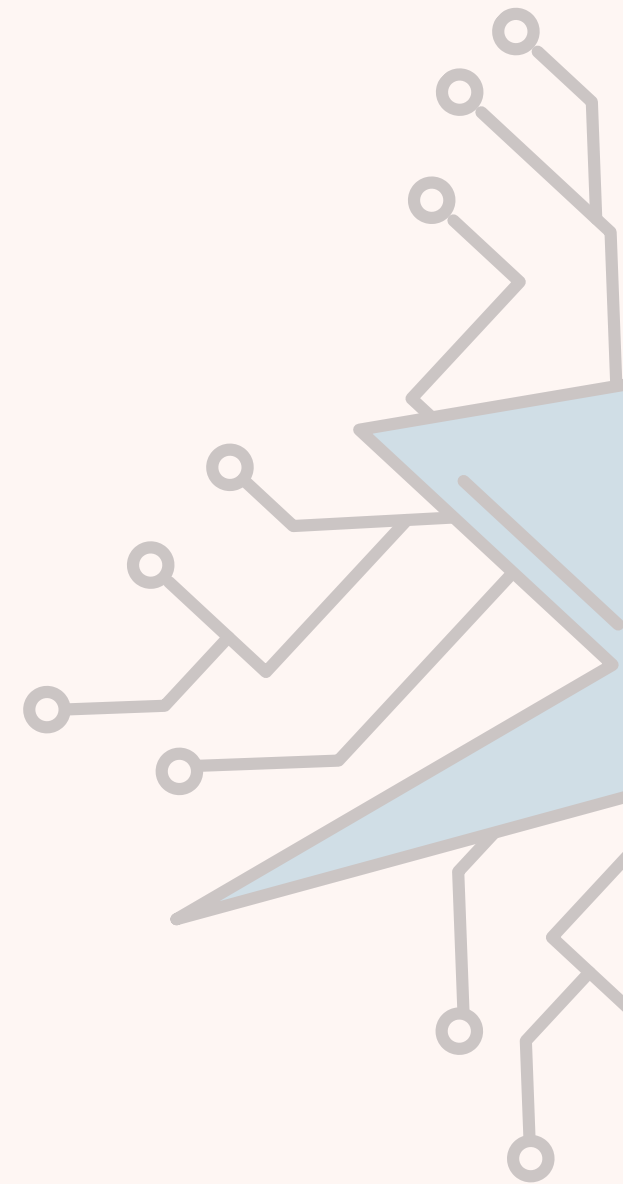
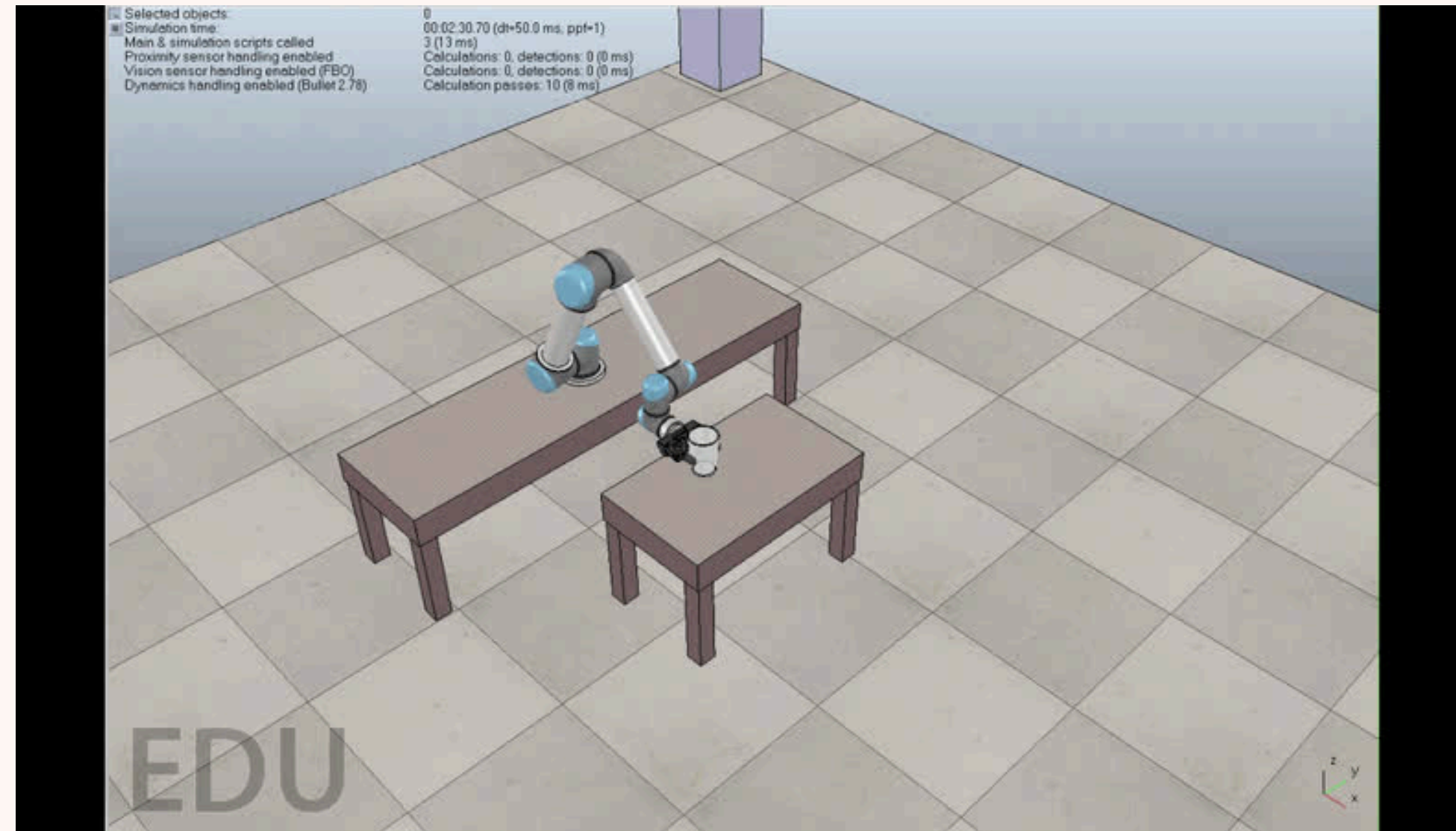
Planejamento de Trajetória

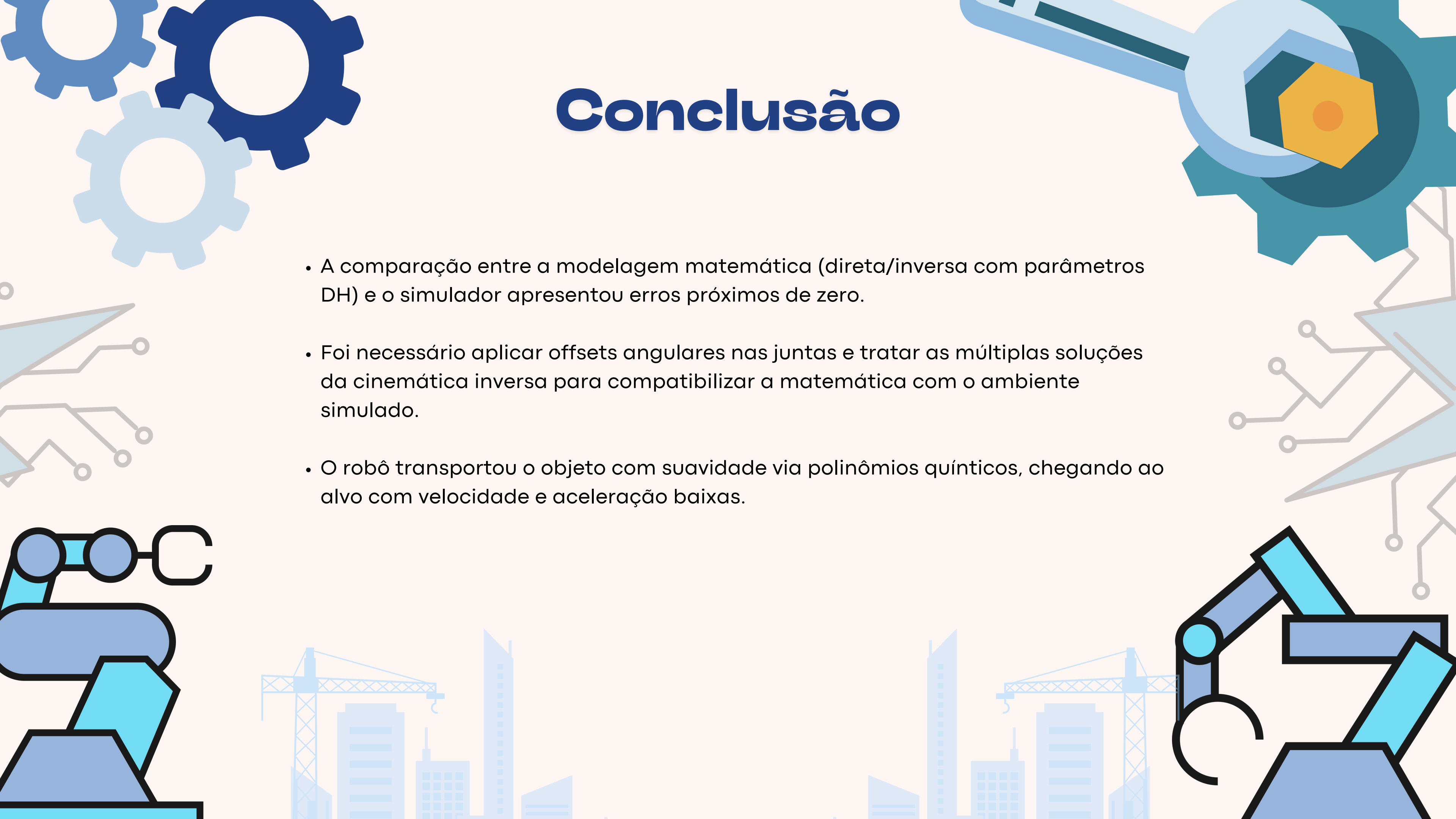
Trajetoária de (30, 10, 30, 15, 30, 45) para (90,-45, 60, 30, -60, 90)



Planejamento de Trajetória

Pick and Place





Conclusão

- A comparação entre a modelagem matemática (direta/inversa com parâmetros DH) e o simulador apresentou erros próximos de zero.
- Foi necessário aplicar offsets angulares nas juntas e tratar as múltiplas soluções da cinemática inversa para compatibilizar a matemática com o ambiente simulado.
- O robô transportou o objeto com suavidade via polinômios quínticos, chegando ao alvo com velocidade e aceleração baixas.



Q&A

Obrigada